

ITAI 13: Intro to Machine Learning

Midterm: Reflection Journal

Group Name: 14274 2_ITAI1371

Team Members:

Marisabel Morales

Joshua Balderas

Katherine Rodriguez

Midterm Reflection Journal

Project Overview and Dataset Selection

For our Midterm project, our team chose the Real Estate dataset from Kaggle.

Initial responsibilities for dataset research and selection were handled collaboratively by Katherine and myself, ensuring we found a dataset that met the professor's requirement of having well over 2,000 rows (our raw dataset contained 25,000 rows and 10 columns).

The primary objective of this project was to establish a clean, preprocessed foundation using Python and Jupyter Lab without making any manual changes to the raw data files. Looking ahead to our final project, this dataset will serve as the backbone for building and training supervised machine learning regression models to predict property Sale Amounts.

Collaborative Workflow and Task Distribution

To ensure a smooth execution of the project, our group divided tasks based on individual strengths and collaborative planning:

- **Dataset Research & Approval:** Katherine and I researched, evaluated, and selected the Kaggle Real Estate dataset, presenting it successfully for formal approval.
- **Coding & Preprocessing Workflow:** Marisabel and I (Joshua) collaborated heavily on the technical implementation within Jupyter Lab. Because the dataset was large, I had selected 5 cities that would shrink our dataset like the professor advised us. After that we developed the complete pipeline covering the initial train-test split, missing value imputation, normalization, feature scaling, One-Hot Encoding, and Matplotlib visualizations.

- **Documentation & Journals:** Katherine compiled the Contribution Journal, while I wrote and prepared this comprehensive Reflection Journal summarizing our technical learnings and workflow.

Key Technical Learnings and Preprocessing Steps

Train-Test Splitting and Data Isolation

One of the most critical lessons reinforced during this project is the strict necessity of partitioning our data prior to any Exploratory Data Analysis (EDA) or preprocessing. Using `train_test_split`, we divided our raw data into a 70% training set (17,500 rows) and a 30% testing set (7,500 rows). We immediately saved and isolated the testing set (`raw_test_data.csv`), ensuring it remained entirely untouched throughout the entire EDA and cleaning process. This prevented data leakage and simulated a true real-world machine learning pipeline where future test data remains completely blind to the modeler.

Exploratory Data Analysis (EDA) on Training Data

All EDA was conducted exclusively on the training dataset. By utilizing Python functions like `info()`, `describe()`, and `isnull().sum()`, we uncovered several data quality issues:

- **Missing Values:** The Property Type column had 6,576 missing values, and Residential Type had 6,797 missing values.
- **Invalid Numerical Entries:** Initial statistical summaries revealed 23 records with an invalid Sale Amount of zero or less, and 117 records with an invalid Assessed Value of zero.
- **Duplicates:** We verified duplicate row counts using `duplicated().sum()`.

Data Cleaning and Imputation

Complying strictly with the rule that no manual changes can be made in Excel, we handled all cleaning programmatically in Python besides the filter of taking 5 cities from said dataset:

- We dropped rows with invalid or missing Sale Amount values.
- For Assessed Value, we converted zero values to NaN and imputed them using the training set's median value (138,320.0) to avoid skewing from extreme outliers.
- Categorical missing values in columns like Town, Property Type, and Residential Type were systematically filled using appropriate categorical defaults ("Unknown").

Feature Engineering and Feature Selection

To extract more predictive power from our text and timestamp fields, we performed targeted feature engineering:

- The original Date Recorded string column was converted to datetime objects, allowing us to extract explicit Sale Year and Sale Month numerical features.
- We performed feature selection by dropping columns that were either unnecessary identifiers (Serial Number), text-heavy and unmanageable (Address), redundant (Date Recorded), or prone to causing severe data leakage (Sales Ratio, which is mathematically derived directly from the sale amount).

Normalization, Scaling, and Visualization

Because property values and financial datasets frequently exhibit heavy right-skewness, we implemented data transformations:

- **Log Normalization:** We applied a log transformation (`np.log1p`) to the Assessed Value column to normalize its distribution shape.
- **Feature Scaling:** We utilized scikit-learn's StandardScaler to standardize the normalized log values, shifting the mean to 0 and standard deviation to 1.
- **Visualizations:** Using Matplotlib, we generated side-by-side histograms demonstrating the clear visual progression of our data distributions before preprocessing, after log normalization, and after standard scaling.

One-Hot Encoding

To prepare our categorical features, specifically Town, Property Type, and Residential Type, for machine learning algorithms that require numerical inputs, we applied One-Hot Encoding using `pd.get_dummies()` with `drop_first=True` to prevent multicollinearity. This expanded our feature matrix cleanly while maintaining data integrity, resulting in a finalized clean training dataset of 17,477 rows and 188 fully numeric columns.

Conclusion

This Midterm project provided invaluable, hands-on experience in turning a messy, raw real-world dataset into a pristine, model-ready format using Python and Jupyter Lab. By dividing responsibilities effectively, maintaining rigorous data isolation between training and testing sets, and executing every transformation programmatically, our group has established a solid, reliable foundation for the machine learning modeling phase in our upcoming finals.